Caching Strategy Handbook

Engineering Handbook · Internal · Owner: Platform Engineering

Purpose

We front hot reads with Memorystore/Redis to protect Spanner and keep api-service latency low. A cache that is mis-sized, mis-expired, or mis-invalidated does the opposite: it collapses under load and stampedes the database. This page is the prescriptive how-to for adding or operating a Redis cache. It exists to prevent eviction storms that dump load onto the database when Redis hits its memory ceiling (BUG-1038), thundering-herd stampedes when a hot key expires (BUG-1039), and stale reads because writes never invalidate the cache (BUG-1040).

Standards

Right-size memory and pick an eviction policy on purpose

When Redis reaches `maxmemory` it evicts according to `maxmemory-policy`. The default `noeviction` makes writes fail; a naive policy evicts your hottest keys and every miss falls through to the database (BUG-1038). Size the instance for the real working set plus headroom, and choose `allkeys-lru` (or `allkeys-lfu` for skewed access) so the least-useful keys go first. Always bound entries with a TTL so the working set cannot grow unbounded, and scale out (add shards / a larger tier) *before* you hit the ceiling — watch `used_memory` and `evicted_keys`.

```
# Memorystore config knobs
maxmemory-policy = allkeys-lru
maxmemory-gb     = 5          # sized to working set + headroom
# Alert when nearing the ceiling
redis-cli INFO memory | grep used_memory_rss
redis-cli INFO stats  | grep evicted_keys   # should stay ~0 in steady state
```

Prevent the thundering herd

When a popular key expires, every concurrent request misses and recomputes it at once, hammering the backend (BUG-1039). Defend with three techniques together: add **jitter** to TTLs so keys do not expire in lockstep; use a **single-flight lock** so exactly one caller recomputes while the rest wait or serve stale; and serve **stale-while-revalidate** where slightly old data is acceptable.

```
import random

def get_or_compute(r, key, compute, ttl=300):
   val = r.get(key)
   if val is not None:
       return val
   lock = r.set(f"lock:{key}", "1", nx=True, ex=10)  # single-flight
   if not lock:
       time.sleep(0.05)                              # let the winner fill it
       return r.get(key) or compute()
   val = compute()
```

```
    r.set(key, val, ex=ttl + random.randint(0, 60))   # jittered TTL
    r.delete(f"lock:{key}")
    return val
```

Invalidate on write; TTL is a safety net

An entry with no TTL that is never invalidated will serve outdated values forever after an update (BUG-1040). On every write, either update the cache (write-through) or delete the key (delete-on-write) so the next read repopulates it, and set a TTL so any missed invalidation still self-heals within a bounded window. Namespace and version keys so a schema change cannot serve an old shape.

```
def update_item(item):
    db.update(item)                       # source of truth first
    r.delete(f"item:v2:{item.id}")        # delete-on-write; next read refills
    # or write-through: r.set(f"item:v2:{item.id}", serialize(item), ex=600)
```

Common failures and fixes

- Hit rate collapses and DB load spikes at peak → right-size memory, set `maxmemory-policy=allkeys-lru`, add TTLs, and scale before the ceiling (BUG-1038).
- Backend hammered when a hot key expires → single-flight lock + jittered TTLs + stale-while-revalidate (BUG-1039).
- Users see outdated values after an update → delete-or-update the cache on write and set a TTL as a safety net (BUG-1040).
- Two features read each other's values → namespace and version keys (e.g. `item:v2:{id}`) and centralize key construction.

Checklist

- Instance sized to working set + headroom; `used_memory` and `evicted_keys` alerted.
- `maxmemory-policy` chosen deliberately (`allkeys-lru`/`allkeys-lfu`), never `noeviction` for a cache.
- Every cache entry has a TTL; hot keys use jittered TTLs.
- Expensive recomputes guarded by a single-flight lock or request coalescing.
- Writes invalidate or update the cache; TTL backs up any missed invalidation.
- Keys are namespaced and versioned; construction is centralized.

Related incidents

BUG-1038, BUG-1039, BUG-1040